DON BOSCO INSTITUTE OF TECHNOLOGY

KURLA , MUMBAI - 400070

A PROJECT REPORT ON

"IoT Based Smart Agriculture Monitoring System"

SUBMITTED BY:

Harshal Bhosale (07),
Rakesh Chaudhary (13),
Pranit Chiman (15),
Om Darekar (73)

UNDER THE GUIDANCE OF

Guide Name: Janhavi Baikerikar

DEPARTMENT OF INFORMATION TECHNOLOGY

(2026-2027)

CERTIFICATE

This is to certify that the project entitled “IoT Based Smart Agriculture Monitoring System” is a bonafide work of Harshal Bhosale (07), Rakesh Chaudhary (13), Pranit Chiman (15), and Om Darekar (73), submitted to the University of Mumbai in partial fulfillment of the requirements of B.E. (Information Technology), Semester VII, for the subject “Internet of Everything (IOE)”.

(GUIDE SIGNATURE) (HOD SIGNATURE)

Dissertation Approval Certificate

This project report entitled “IoT Based Smart Agriculture Monitoring System” by Harshal Bhosale, Rakesh Chaudhary, Pranit Chiman, and Om Darekar is approved for the degree of Bachelor of Engineering in Information Technology.

Examiners

1. _____

2. _____

Name:-

Date:-

Place:-

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1. Harshal Bhosale - 07
2. Rakesh Chaudhary - 13
3. Pranit Chiman - 15
4. Om Darekar - 73

Date:

ABSTRACT

Agriculture is an important sector of the Indian economy, and efficient farming depends on timely information about soil and environmental conditions. Traditional field inspection can delay responses to inadequate soil moisture, excessive temperature, high humidity, and potentially hazardous gases. A low-cost monitoring system can support better resource management by making these measurements available to users in a clear and accessible form.

This Semester VII Internet of Everything (IOE) project implements an IoT Based Smart Agriculture Monitoring System using an ESP32 development board. A DHT11 sensor measures ambient temperature and relative humidity, a resistive soil moisture probe supplies an analog soil reading, and an MQ7 module supplies an analog gas sensor reading. The ESP32 acquires data from these sensors and provides it over its built-in Wi-Fi connection.

Unlike the previous semester implementation, which used the Blynk mobile application, the current system uses a custom HTML, CSS, and JavaScript web dashboard. An ESP32 HTTP web server exposes a JSON data endpoint at /data. The browser requests readings periodically and displays gauges, trends, connectivity status, and a summary of environmental conditions. The dashboard also retrieves a near-term weather forecast directly from OpenWeatherMap using a user-provided API key.

The decision-support logic compares soil moisture, temperature, humidity, and the raw MQ7 signal with configurable limits. For dry soil, it can advise irrigation or recommend waiting when the forecast indicates a high chance of rain. It also produces warnings for waterlogging, heat stress, elevated gas readings, and combined heat-humidity conditions. Soil moisture is calculated from user-adjustable dry and wet calibration values; gas measurements remain raw ADC values and are not presented as calibrated carbon-monoxide concentrations.

The result is a local-network monitoring and decision-support prototype that connects physical sensors, an embedded web service, live browser visualizations, and external weather information. It does not automatically operate pumps or ventilation equipment. Future improvements can include authenticated remote access, validated sensor calibration, historical data storage, and actuator-based control. The implementation demonstrates the practical use of interconnected devices and information services in Internet of Everything applications.

CONTENTS

Sr. No.	Title	Page No.
1.	Introduction	1
1.1	Introduction	1
1.2	Literature Survey	1
1.3	Problem Statement and Objectives	4
1.4	Proposed Solution	5
1.5	Wireless Technology Used	6
2.	Analysis and Design	8
2.1	Methodology	8
2.2	System Architecture / Design	9
2.3	Flow Chart	9
2.4	Circuit Diagram	11
2.5	Hardware Requirements	12
2.6	Software Requirements	16
2.7	Cost Estimation	17
3.	Implementation	20
3.1	Implementation, Snapshots /figure with explanation	20
3.2	Code	24
3.3	Future direction	27
4.	Conclusion	28
5.	References	29

LIST OF FIGURES

Fig. No.	Title
Fig. 1	Flow chart
Fig. 2	Circuit diagram
Fig. 3	ESP32 development board
Fig. 4	DHT11 temperature and humidity sensor
Fig. 5	Soil moisture sensor
Fig. 6	MQ7 gas sensor
Fig. 7	Complete hardware setup of the IoT-based smart agriculture monitoring system on a breadboard
Fig. 8	Close-up view of ESP32 interfacing with DHT11, soil moisture, and MQ7 gas sensors
Fig. 9	Custom web dashboard interface (initial connection view)
Fig. 10	Dashboard settings and weather configuration

1. INTRODUCTION

1.1 Introduction

The Internet of Everything (IOE) extends connected-device concepts by bringing together people, processes, data, and things to make information more useful. In agriculture, environmental sensors can capture field conditions, microcontrollers can publish readings, and a user interface can transform these readings into practical recommendations. Such systems support informed irrigation decisions and improve awareness of changing growing conditions.

Traditional agricultural monitoring commonly relies on manual inspection of soil moisture, temperature, humidity, and surrounding air conditions. These checks may be infrequent and can miss rapid changes. Weather conditions also affect irrigation decisions: for example, watering immediately before expected rainfall may be unnecessary. Combining local measurements with forecast information can therefore improve the timing of user decisions.

The proposed system connects three field sensors to an ESP32, which makes the readings available through a local Wi-Fi network. A browser-based dashboard fetches the device data without requiring a Blynk account or a separate cloud database. Its interface displays live values and status indicators, while its recommendation module evaluates conditions against thresholds selected by the user.

The ESP32 is the processing and networking unit. It reads digital data from the DHT11 on GPIO4 and analog data from the soil moisture probe and MQ7 module on GPIO34 and GPIO35. The firmware is written using the Arduino IDE and uses the WiFi, WebServer, and DHT libraries. A simple HTTP response containing JSON is accessible at the /data endpoint for the dashboard to read.

The web dashboard is developed using HTML, CSS, and JavaScript. It provides sensor value cards, small trend charts, a connection indicator, and decision-support messages. It also calls the OpenWeatherMap forecast API directly from the browser when the city and weather key are configured. Configurable soil calibration values and warning limits are retained in the browser's local storage.

Together, these components illustrate IOE integration across things (sensors and ESP32), data (JSON measurements and forecasts), processes (threshold evaluation), and people (actionable advice displayed in a browser). The present deployment requires the user's browser to reach the ESP32 on the same or otherwise accessible local network; wider internet access would require additional networking and security measures.

1.2 Literature Survey

Research and technical documentation on smart agriculture highlight the value of interconnected environmental sensors, inexpensive microcontrollers, and accessible decision interfaces. This section summarizes technologies relevant to the implemented system, with particular attention to Wi-Fi sensing, browser-based data display, and incorporation of short-term forecast information. Performance claims have not been added without measured evidence from this prototype.

1.2.1 IoT and IOE in Agriculture — Overview

IoT applications capture measurements from physical devices and communicate them to software systems. In an IOE-oriented approach, the same measurements can be combined with external information and user-defined processes to support human decisions. In this project, local sensor data and an online forecast are considered together when generating recommendations.

Sensor-based systems can help users identify dry soil, excessive moisture, and unfavorable environmental conditions earlier than occasional manual inspection. However, benefits such as water savings or increased crop yield depend on deployment conditions and must be verified experimentally. This prototype demonstrates monitoring and rule-based recommendations rather than a measured improvement in agricultural yield.

1.2.2 ESP32 as a Microcontroller

The ESP32 microcontroller, developed by Espressif Systems, is widely used in IoT applications due to its high performance and built-in connectivity features. It integrates both WiFi and Bluetooth capabilities, making it suitable for real-time data transmission.

Key features of ESP32 include:

- Dual-core Xtensa LX6 processor (up to 240 MHz)
- Integrated WiFi (802.11 b/g/n) and Bluetooth (BLE)
- 12-bit Analog-to-Digital Converter (ADC)
- Multiple GPIO pins for sensor interfacing
- Low power consumption modes

The ESP32 is appropriate for this project because it can operate a small HTTP server while acquiring sensor data. Its ADC1-capable GPIO34 and GPIO35 pins are used for analog measurements, and the built-in wireless interface allows a browser on the network to retrieve structured data.

1.2.3 DHT11 Sensor

The DHT11 is an economical digital temperature and humidity sensor that provides readings through a single digital data pin. In the supplied firmware, it is initialized on GPIO4, with readings refreshed at approximately two-second intervals.

Its specifications include:

- Temperature range: 0°C to 50°C (±2°C accuracy)
- Humidity range: 20% to 90% RH (±5% accuracy)

The DHT11 offers basic ambient monitoring, but its specified accuracy and operating

range limit its suitability for precise agricultural control. If a reading fails, the ESP32 sets the dhtOk flag to false and retains the previous temperature and humidity values, which the dashboard identifies as a sensor-read problem.

1.2.4 Soil Moisture Sensor

The soil moisture probe produces an analog signal that changes with the electrical properties of soil. The current ESP32 implementation samples this signal on GPIO34. The dashboard interprets higher raw readings as drier soil and converts them to an indicative percentage using configurable readings for dry soil and wet conditions.
There are two main types:

- **Resistive sensors**, which measure electrical resistance between probes
- **Capacitive sensors**, which measure dielectric properties and are more durable

The conversion depends on calibration and soil composition, so the displayed percentage is an estimated relative moisture index rather than a laboratory measurement of volumetric water content. The default dry and wet values in the dashboard can be replaced with values observed from the installed sensor.

1.2.5 MQ7 Gas Sensor

The MQ7 is a semiconductor gas sensor commonly used for carbon monoxide sensing. In this prototype, its analog output is sampled using ESP32 GPIO35. The browser displays the measured ADC value as a raw gas level, allowing threshold-based warnings but not claiming a calibrated CO concentration.
Typical uses include:

- Indoor air quality monitoring
- Greenhouse gas detection
- Industrial safety systems

Reliable CO measurements would require the sensor manufacturer's recommended heater-cycling procedure, calibration, and suitable analog signal conditioning. The present firmware does not implement these steps. Therefore, its gas-level alerts should be treated as prototype indications rather than certified safety or air-quality measurements.

1.2.6 Web Dashboard and Weather API

A browser-based dashboard allows field readings to be viewed without a proprietary mobile application. The ESP32 acts as a local HTTP server and the browser periodically requests JSON readings from /data. HTML provides the page structure, CSS styles the interface, and JavaScript handles polling, visualization, status messages, and decision rules.
The current dashboard supports the following features:

- Live temperature, humidity, soil moisture, and raw gas displays with trend indicators
- Local Wi-Fi connection status, automatic refreshing, and configurable sensor thresholds
- Direct OpenWeatherMap forecast requests from the browser for a selected city
- Advice on irrigation, rain, heat, gas levels, and high temperature with humidity

The weather API key is entered and saved in the browser, not stored securely on a backend. This architecture is simple for a prototype but should use safer credential handling and authenticated device access before any public deployment.

1.3 Problem Statement and Objectives

1.3.1 Problem Statement

Farmers may have limited continuous information about soil wetness and microclimatic conditions. When decisions rely only on occasional observation, irrigation can be mistimed, and environmental changes can remain unnoticed. A practical solution must present local readings in an understandable form and make them easy to refresh.

Over-irrigation and under-irrigation are common concerns. Soil moisture readings help users identify both dry and excessively wet conditions; however, the need for irrigation also depends on upcoming rainfall. Temperature and humidity monitoring support environmental awareness, while gas-sensor measurements can indicate changes that deserve attention.

Many ready-made IoT monitoring systems depend on a cloud dashboard or a proprietary mobile application. For the current project, the requirement is to view ESP32 readings directly through a custom web dashboard and incorporate weather information without relying on Blynk. The system should remain economical and usable from a browser on the local network.

The problem addressed is therefore to integrate sensor acquisition, Wi-Fi communication, live visualization, configurable thresholds, and external weather data in one decision-support interface. The system is designed to recommend user actions; automatic irrigation, calibrated gas safety monitoring, and unrestricted internet access are outside the present implementation.

1.3.2 Objectives

The primary objective is to implement an affordable ESP32-based agriculture monitoring and decision-support prototype for the Semester VII IOE subject. Specific objectives are:

- To collect temperature, relative humidity, soil moisture, and raw gas data using DHT11, soil moisture, and MQ7 sensors.
- To connect the sensors to the ESP32 using a digital input and ADC1 analog inputs.
- To host sensor data on an ESP32 HTTP server and make it available as JSON at /data over Wi-Fi.
- To design a responsive web dashboard for displaying readings and connection status.
- To estimate soil moisture percentage using adjustable dry/wet calibration readings.
- To combine sensor thresholds with a browser-retrieved weather forecast for user recommendations.
- To support local serial debugging and provide a basis for later logging, security, and automated irrigation.

1.4 Proposed Solution

The project implements an IoT Based Smart Agriculture Monitoring System that combines real-time sensor monitoring and browser-based decision support. It is built around an ESP32 development board connected to three environmental sensor modules.

The DHT11 on GPIO4 provides temperature and humidity values. The analog soil moisture probe on GPIO34 supplies a raw reading used to estimate relative moisture, and the MQ7 on GPIO35 provides a raw gas-sensor value. All sensor data is acquired by the ESP32 firmware developed using the Arduino IDE.

The ESP32 connects to Wi-Fi and runs a small HTTP server. A browser requests the /data URL and receives temperature, humidity, soil, gas, and DHT sensor-status fields in JSON format. Instead of the earlier Blynk application, the current HTML/CSS/JavaScript dashboard periodically polls this endpoint and renders the incoming measurements.

The dashboard uses configurable threshold limits to identify dry soil, possible waterlogging, heat stress, elevated raw gas values, and high heat with humidity. It separately calls the OpenWeatherMap forecast API; when likely rain coincides with dry soil, it may advise postponing irrigation rather than watering immediately.

An ESP32 Serial Monitor output supports testing and troubleshooting. The browser indicates unreachable devices and displays a DHT11 warning if the device reports a failed sensor read. Users can edit calibration and decision thresholds through the dashboard settings.

The system is intended for supervised decision support. It does not drive a pump or guarantee the accuracy of uncalibrated gas readings. Expansion options include secure remote access, historical records, formal calibration, and actuator-based control.

1.4.1 Scope of the Project

The current project covers the ESP32-based collection of temperature, humidity, soil moisture and raw gas-sensor readings; local Wi-Fi data access through an HTTP/JSON endpoint; real-time visualization on a browser dashboard; and basic advisory decisions using user-defined sensor limits and an external weather forecast. The system is intended as a monitoring and decision-support prototype. Automated irrigation control, long-term cloud data storage, unrestricted remote access and calibrated carbon-monoxide measurement are outside its present scope.

1.4.2 Sustainable Development Goals (SDGs)

The project relates to the following United Nations Sustainable Development Goals:

SDG 2 - Zero Hunger: Monitoring field conditions can support better-informed crop care and timely irrigation decisions, contributing to the objective of sustainable agriculture.

SDG 6 - Clean Water and Sanitation: Soil moisture readings combined with rainfall forecasts can help users avoid unnecessary watering and encourage efficient water use.

SDG 9 - Industry, Innovation and Infrastructure: The ESP32, Wi-Fi sensor connectivity, custom web dashboard and weather API demonstrate an affordable digital monitoring application.

SDG 12 - Responsible Consumption and Production: Condition-based recommendations encourage more careful use of water and other agricultural resources instead of relying only on routine schedules.

SDG 13 - Climate Action: Forecast-aware recommendations can help users respond to short-term rainfall and heat conditions while supporting awareness of changing local weather.

These SDG alignments describe the project's intended contribution; field-level savings, improved crop yields and climate outcomes have not been experimentally established.

1.5 Wireless Technology Used

Wi-Fi is the main wireless technology used to connect the ESP32 sensor node with the browser-based dashboard. Weather information is requested separately over the internet by the browser.

The ESP32 provides built-in 2.4 GHz Wi-Fi connectivity and runs in station mode after joining the configured wireless network. No additional gateway or Blynk service is needed for collecting sensor readings from its built-in HTTP server.

After connecting to the network, the ESP32 displays its assigned IP address in the Serial Monitor and starts a WebServer on port 80. The dashboard sends ordinary HTTP GET requests to http://<ESP32-IP>/data, which returns structured JSON data. This connection is local unless additional routing or secure remote-access infrastructure is configured.

The data communication process consists of:

- Reading the sensor values on the ESP32 and exposing them at the /data endpoint
- Fetching the JSON response from the browser at an adjustable refresh interval (default: 3 seconds)
- Requesting the external weather forecast over HTTPS directly from the browser when configured

Wi-Fi was selected because of the following practical advantages:

- The ESP32 can join an existing local network without an external radio gateway
- HTTP and JSON are straightforward to inspect, test, and integrate in a web page
- Browser access is available on many computers and smartphones on the network
- A custom dashboard can operate without a proprietary IoT cloud subscription.

This design depends on reliable Wi-Fi and permits access to a local HTTP endpoint without authentication. A publicly hosted HTTPS page may also be blocked from fetching local HTTP data because of browser mixed-content rules. Production deployment would require network isolation, secure transport, and controlled access.
The combination of local Wi-Fi sensing and an independent HTTPS weather API illustrates how internal device information and external internet services can be integrated in an IOE application.

2. ANALYSIS AND DESIGN

2.1 Methodology

Development followed a structured method involving component selection, sensor wiring, embedded firmware, web dashboard development, API integration, calibration settings, and functional review.

1. System Design

Required inputs were identified as ambient temperature, humidity, soil moisture, and the raw gas-sensor signal. The ESP32 was retained as the central controller because it provides Wi-Fi, digital GPIO, and ADC inputs suitable for the selected sensors.

2. Hardware Implementation

The DHT11 was wired to ESP32 GPIO4, the soil moisture analog output to GPIO34, and the MQ7 analog output to GPIO35. All devices share ground, and sensor supply voltages must be checked against their module specifications. If the MQ7 module is powered at 5 V, its analog output must be kept within the ESP32's 3.3 V input limits, such as by using a suitable voltage divider.

3. Software Development

The firmware was written in Arduino C/C++ using WiFi.h, WebServer.h, and DHT.h. It connects to the configured Wi-Fi network, starts an HTTP server, updates cached sensor readings periodically, and returns the latest values in JSON. The browser dashboard is implemented separately using HTML, CSS, and JavaScript.

4. Data Acquisition and Processing

The DHT11 provides digital temperature and humidity values. Soil and gas inputs are each averaged over ten analog samples in the ESP32 firmware; the readings are refreshed about every two seconds. Soil moisture is converted to a percentage in the browser using dry and wet calibration values; MQ7 readings are displayed as raw ADC units rather than ppm.

5. Local HTTP Data Service

The ESP32 publishes a JSON document at /data through its local web server. The payload includes temperature, humidity, soil, gas, and dhtOk. A Cache-Control header avoids stale responses, and an Access-Control-Allow-Origin header enables the separate web dashboard to request the endpoint during prototyping.

6. User Interface Development

The responsive web interface displays values, mini-trend plots, an overall status banner, and recommended actions. It polls the ESP32 at a default interval of three seconds. The same browser independently retrieves four upcoming three-hour forecast records from OpenWeatherMap and estimates the greatest rain probability and forecast temperature.

7. Testing and Calibration

The implementation exposes configurable dry/wet calibration readings and warning limits for soil moisture, temperature, and raw gas values. The provided files do not contain a formal measurement dataset or calibration certificate, so numerical sensor accuracy and field performance are not asserted. Functional validation should compare browser readings with Serial Monitor data.

8. System Operation

On start-up, the ESP32 joins Wi-Fi, prints its IP address, starts serving /data, and reads its sensors. The dashboard connects after a user enters that IP address and chooses Connect. The browser can refresh readings, display status, and update decisions as readings or weather data change.

9. Validation of Results

A source-code review verifies the intended data path, endpoint structure, sample timing, and decision rules. Full on-site test results and calibrated reference readings are not included in the supplied materials; consequently, the report describes the implemented design and its testable operation without inventing measurement outcomes.

2.2 System Architecture / Design

The architecture follows the sensing, processing, communication, and decision-support layers of an IOE system.

At the sensing layer, DHT11 measures temperature and humidity, the soil moisture sensor provides an analog wetness signal, and the MQ7 module provides an analog gas signal. Each sensor connects directly to the ESP32.

At the processing layer, the ESP32 samples sensor outputs, averages the analog values, retains the latest readings, and creates a compact JSON response. It also reports whether the DHT11 read succeeded.

At the communication layer, the ESP32 uses the local Wi-Fi network to serve http://<ESP32-IP>/data. The browser requests this endpoint periodically; separately, the browser fetches a forecast from OpenWeatherMap over HTTPS. There is no Blynk cloud, MQTT broker, or project database in the current implementation.

At the application layer, JavaScript converts raw soil readings into a percentage and compares the measurements with user-configured limits. A forecast rain chance of at least 60% can change the dry-soil recommendation from irrigation to waiting. Warnings also cover high heat, possible waterlogging, and high gas readings.

The user sees measured values, recent trends, an overall status, and action-oriented messages on the dashboard. This arrangement connects physical devices, data services, decision rules, and users without directly controlling any agricultural actuator.

2.3 Flow Chart

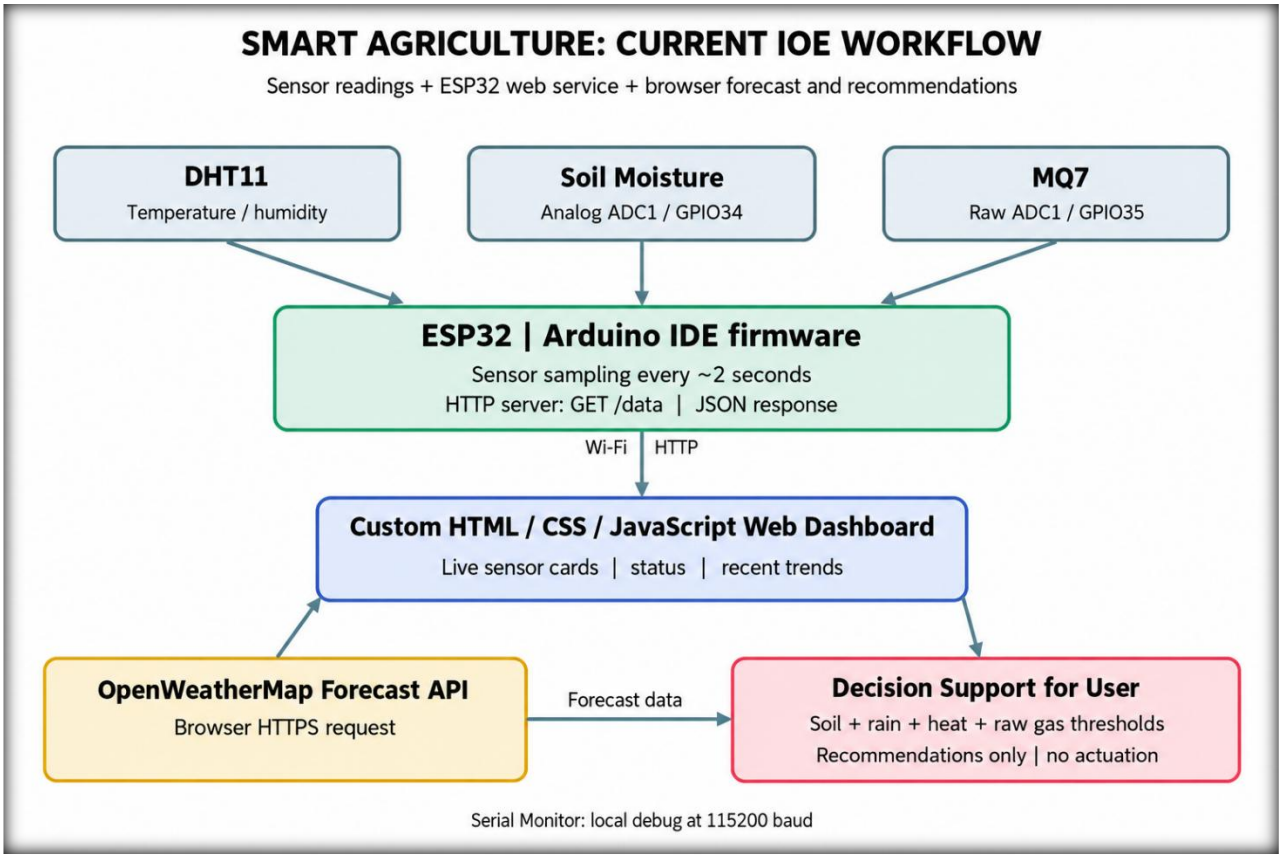


Fig. 1. Flow Chart

Figure 1 describes the path from sensor acquisition to the browser dashboard and its decision-support output.

The DHT11 supplies temperature and humidity, while the soil moisture sensor and MQ7 module supply analog readings. The ESP32 uses GPIO4 for DHT11 data, GPIO34 for the soil input, and GPIO35 for the gas input.

The microcontroller refreshes readings approximately every two seconds. Ten samples are averaged for each analog channel. The cached values and DHT status are formatted as JSON and made available through the built-in HTTP web server.

The browser requests /data over the local Wi-Fi network at the selected refresh interval, which defaults to three seconds. The JSON fields populate the temperature, humidity, soil moisture, and raw gas cards.

The browser separately requests a near-term OpenWeatherMap forecast. JavaScript combines the latest sensor readings, editable thresholds, and forecast conditions to produce decision-support advice. No remote Blynk service is used.

The ESP32 also writes readings to the Arduino Serial Monitor at 115200 baud for local inspection and debugging.

The complete workflow illustrates sensor-to-browser information transfer and the transformation of device readings into practical guidance for the user.

2.4 Circuit Diagram Description

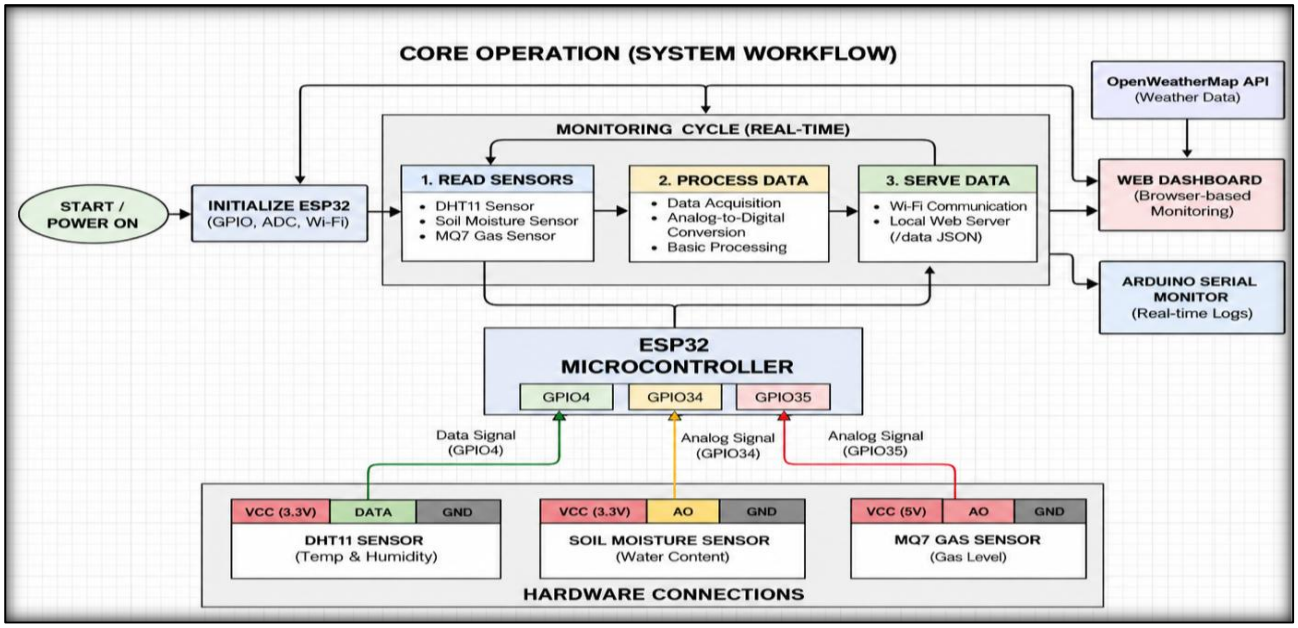


Fig. 2. Circuit Diagram

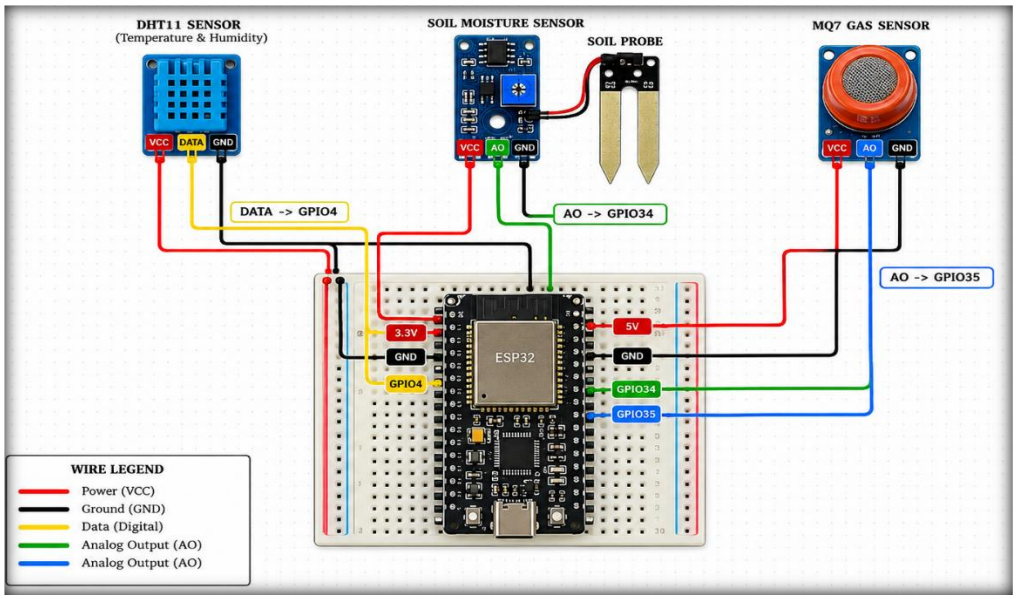


Fig. 2.1. Circuit Diagram

The circuit diagram shows the ESP32 connected to the DHT11, soil moisture probe, and MQ7 module. The hardware connections remain the same as the previous semester implementation; the communication and dashboard software have changed.

The **DHT11 sensor** is connected to the ESP32 for measuring temperature and humidity. Its data pin is connected to **GPIO4**, while the VCC and GND pins are connected to the 3.3V power supply and ground respectively. The sensor provides digital output directly to the microcontroller.

The **Soil Moisture Sensor** is used to measure the water content in the soil. Its analog output (AO) pin is connected to **GPIO34**, which is an analog input pin of the ESP32. The sensor is powered using the 3.3V supply and ground from the ESP32.

The MQ7 module provides an analog output connected to ESP32 GPIO35. The module heater commonly needs a 5 V supply, while the ESP32 ADC input must not exceed its

permitted voltage range. Correct module power, common grounding, and voltage-level protection must be verified before operation.
The ESP32 reads the digital DHT11 output and both analog sensor channels. It then makes the latest data available through its local HTTP server, rather than transmitting values to the Blynk Cloud.
Overall, the circuit design is simple, efficient, and suitable for real-time environmental monitoring in smart agriculture applications.

2.5 Hardware Requirements

The hardware components used in this project form the foundation of the IoT-based smart agriculture monitoring system. These components are responsible for sensing environmental parameters, processing the data, and enabling wireless communication with the cloud platform. The selection of each component was based on reliability, compatibility with the ESP32 microcontroller, and suitability for real-time monitoring applications.

2.5.1 ESP32 Development Board

Property	Details
Manufacturer	Espressif Systems
CPU	Dual-core Xtensa LX6, up to 240 MHz
Flash Memory	4 MB
RAM	520 KB SRAM
WiFi	802.11 b/g/n (2.4 GHz)
Bluetooth	Bluetooth 4.2 + BLE
ADC Resolution	12-bit (0–4095)
Operating Voltage	3.3V logic, powered via 5V USB
GPIO Pins	34 programmable GPIO pins
ADC Channels	18 channels (12-bit SAR ADC)

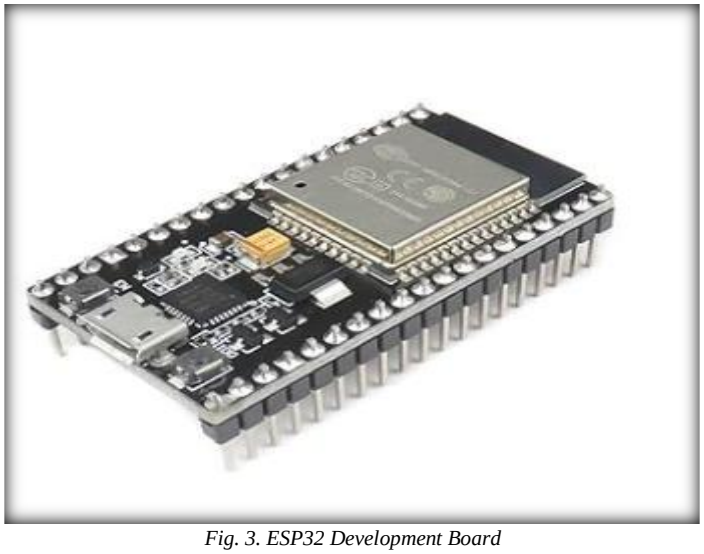


Fig. 3. ESP32 Development Board

The ESP32 development board acts as the sensing and communication controller. It reads DHT11 data on GPIO4 and analog soil/gas signals on GPIO34 and GPIO35, handles the device Wi-Fi connection, and serves measurements from a built-in HTTP endpoint. Arduino IDE is used to program the board.

2.5.2 DHT11 Temperature and Humidity Sensor

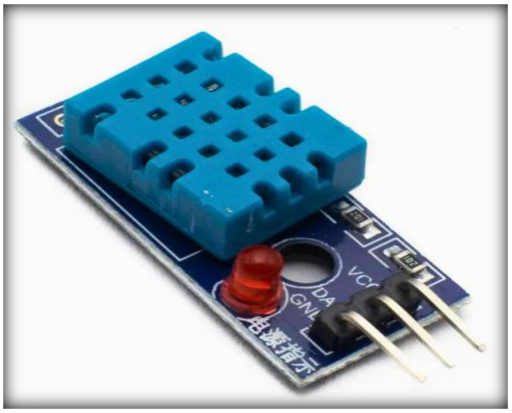


Fig. 4. DHT11 Temperature and Humidity Sensor

Property	**Details**
Temperature Range	0°C to 50°C
Temperature Accuracy	±2°C
Humidity Range	20% to 90% RH
Humidity Accuracy	±5% RH
Communication Type	Single-wire digital
Operating Voltage	3.3V – 5V
Sampling Rate	1 reading per second
Connected Pin	GPIO 4 (ESP32)

The DHT11 sensor is used to measure ambient temperature and humidity. It operates using a proprietary single-wire communication protocol to transmit a 40-bit data packet containing calibrated digital values. The sensor plays a critical role in monitoring climatic conditions that directly affect crop growth. The integration with the ESP32 is simplified using the DHT library, which handles low-level communication and provides user-friendly output in floating-point format.

2.5.3 Soil Moisture Sensor

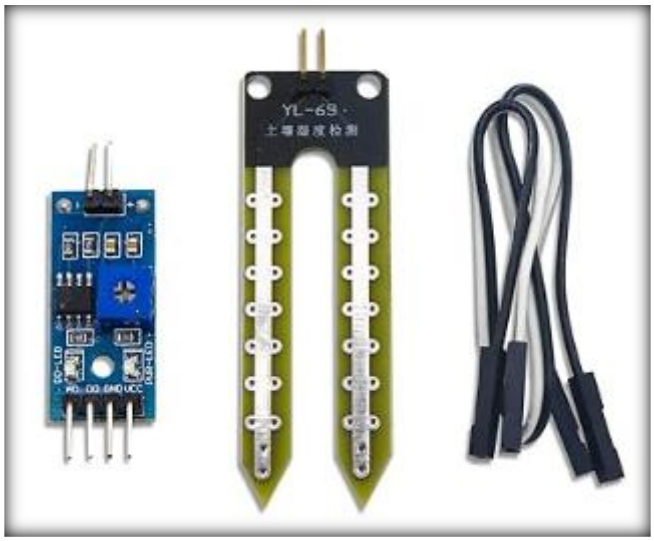


Fig. 5. Soil Moisture Sensor

Property	Details
Sensor Type	Resistive (Analog Output)
Output	Analog Voltage (0V – VCC)
ADC Input Range	0 – 4095 (12-bit ESP32 ADC)
Operating Voltage	3.3V
Connected Pin	GPIO 34 (Input-only ADC pin)
Output (Dry Soil)	~4095 (High resistance)
Output (Wet Soil)	~0 (Low resistance)

The resistive soil moisture probe is sampled on GPIO34. The dashboard computes an estimated percentage using the configurable dry and wet ADC readings (default values: 3200 and 1400). Values below 30% request irrigation advice, while values above 85% indicate possible waterlogging, unless thresholds are edited.

2.5.4 MQ7 Gas Sensor

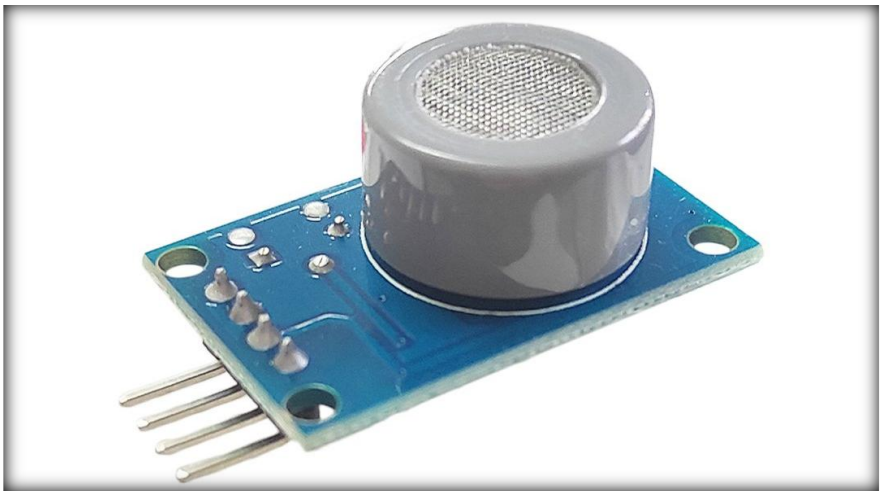


Fig. 6. MQ7 Gas Sensor

Property	Details
Target Gases	Primarily carbon monoxide (CO); other gases may affect response
Measurement Display	Raw ESP32 ADC value; not calibrated ppm
Output	Analog Voltage
ADC Input Range	0 – 4095 (12-bit ESP32 ADC)
Module Power / ADC Safety	5V heater module; protect ESP32 ADC input from >3.3V
Warm-up / Calibration	Manufacturer-specific; calibrated CO measurements not implemented
Connected Pin	GPIO 35 (Input-only ADC pin)

The MQ7 analog signal is sampled on GPIO35 and shown as a raw 0–4095 ADC reading. The dashboard's default warning and critical thresholds are 1500 and 2500 raw units. These are application settings, not verified CO ppm levels or safety limits; proper heater control, calibration, and electrical protection would be necessary for quantitative use.

2.5.5 Supporting Components

Component	Purpose
Breadboard	Enables circuit prototyping without soldering
Jumper Wires	Used for electrical connections between components
USB Cable (Micro/Type-C)	Used for programming and powering the ESP32
External 5V Power Supply	Provides standalone power during field deployment

Supporting components are essential for assembling and operating the system. The breadboard and jumper wires facilitate easy and flexible connections, while the USB cable is used for both programming and power supply. An optional external 5V supply allows the system to function independently without a computer, making it suitable for real-world agricultural deployment.

2.6 Software Requirements

The software stack includes the Arduino IDE for ESP32 firmware, the ESP32 WiFi and WebServer libraries, the DHT sensor library, a custom HTML/CSS/JavaScript browser dashboard, and the OpenWeatherMap forecast API. A Blynk account or Blynk-specific library is not required.

2.6.1 Arduino Integrated Development Environment (IDE)

The Arduino Integrated Development Environment (IDE) is used for writing, compiling, and uploading the firmware to the ESP32 microcontroller. It provides a user-friendly interface and supports C/C++-based programming, making it suitable for embedded system development.

Feature	Description
Development Language	C/C++
Board Support	ESP32 board package by Espressif Systems (via Board Manager)
Compiler	Built-in GCC compiler
Serial Monitor	Used for real-time debugging at 115200 baud rate
Library Manager	Simplifies installation and management of external libraries

The Arduino IDE enables seamless integration of third-party libraries and provides tools for debugging and monitoring system behavior during runtime.

2.6.2 Libraries and Web Technologies Used

The firmware uses WiFi.h to connect to the local network, WebServer.h to provide the /data endpoint, and DHT.h to read the DHT11. Browser-side HTML, CSS, and JavaScript implement rendering, network polling, settings, and decision support.

Library / Technology	**Purpose**	**Source**
WiFi.h	Connects the ESP32 to local Wi-Fi	ESP32 core
WebServer.h	Serves HTTP requests and JSON data at /data	ESP32 core
DHT.h	Reads digital DHT11 temperature and humidity	Adafruit
HTML / CSS / JavaScript	Renders the responsive browser dashboard and decision rules	Web standards
OpenWeatherMap API	Returns weather forecasts to the browser	OpenWeather

OpenWeatherMap is an external forecast service. It is accessed directly by the browser and is not called by the ESP32 firmware. The local web service and the remote weather service are therefore separate data sources.

2.6.3 Custom Web Dashboard and Weather Integration

The custom dashboard replaces the earlier Blynk mobile application. It connects to the ESP32 by local IP address, fetches the JSON measurements, and provides animated indicators, recent-reading graphs, connectivity status, and recommendations based on user-defined limits.

Feature	**Description**
Device data endpoint	ESP32 HTTP GET /data returns latest sensor readings in JSON
Local sensor access	Dashboard connects using ESP32 IP address on reachable Wi-Fi
Real-time visualization	Animated readings, trend plots, and connection status
Decision support	Rules for moisture, heat, humidity, gas, and forecast rain
Weather integration	Browser directly fetches near-term OpenWeatherMap forecast
Configuration	Browser localStorage retains user calibration and thresholds

The dashboard uses the following configuration values:

Parameter	**Description**
ESP32 IP address	Local address used to request http://<IP>/data
Forecast city	City used to request weather information
Weather API key	Entered in browser settings for OpenWeatherMap

The user enters the ESP32 IP address, forecast city, and an OpenWeatherMap API key through browser inputs. Settings are stored in localStorage. This makes the prototype easy to demonstrate, but a public deployment should protect API credentials and restrict access to the sensor endpoint.

2.7 Cost Estimation

The estimated cost covers the existing ESP32 board, DHT11, soil moisture and MQ7 sensors, and supporting prototype hardware. Replacing Blynk with a custom browser interface does not require an additional paid platform for basic local monitoring.
The approximate cost of each component is listed below:

2.7.1 Hardware Cost Breakdown

Sr. No.	Component	Quantity	Approx. Cost (₹)	Total Cost (₹)
1	ESP32 Development Board	1	500	500
2	DHT11 Temperature & Humidity Sensor	1	100	100
3	Soil Moisture Sensor	1	80	80
4	MQ7 Gas Sensor	1	150	150
5	Breadboard	1	120	120
6	Jumper Wires	1 set	100	100
7	USB Cable (Type-C/Micro USB)	1	80	80
8	External 5V Power Supply (optional)	1	150	150

2.7.2 Total Cost

Category	Amount (₹)
Total Hardware Cost	**₹1280**

2.7.3 Software Cost

The software tools used in this project are available without a mandatory license fee for this prototype:

Software	Cost
Arduino IDE	Free
Custom Web Dashboard (HTML/CSS/JavaScript)	Free
ESP32 and DHT Libraries	Free

2.7.4 Overall Project Cost

Category	**Amount (₹)**
Hardware Cost	1280 ₹
Software Cost	0 ₹
Total Cost	**1280 ₹**

2.7.5 Remarks

The listed hardware cost is an indicative estimate rather than a purchase invoice. The Arduino IDE, browser dashboard, and required libraries are free to use; the external weather service may have plan-based usage restrictions. The design is suited to academic prototyping and supervised field demonstrations.

3. IMPLEMENTATION

3.1 Implementation and Snapshots / Figures with Explanation

The current implementation retains the existing ESP32 sensor wiring and replaces Blynk cloud transmission with an embedded HTTP web server and a browser-based decision dashboard. The diagrams and screenshots show the hardware and user-interface portions of the prototype.

3.1.1 Complete hardware setup of the IoT-based smart agriculture monitoring system on a breadboard.

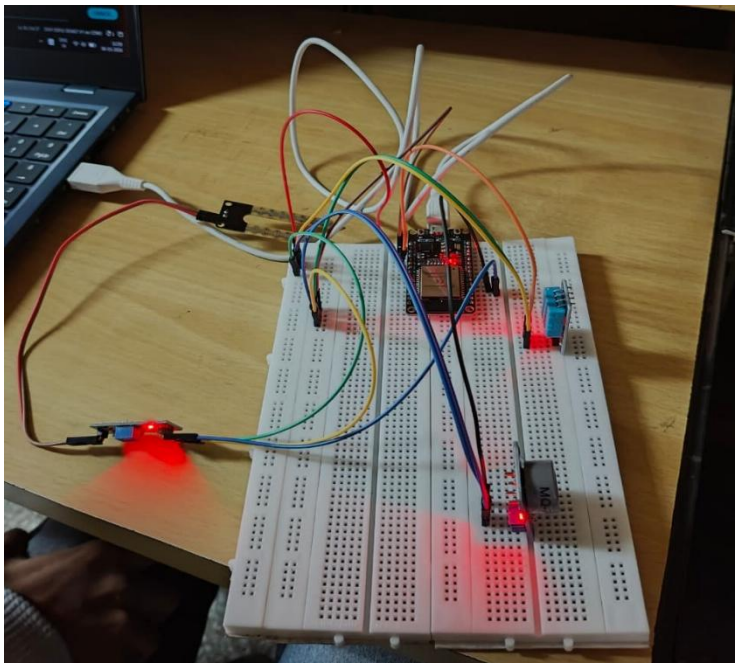


Fig. 7. Complete hardware setup of the IoT-based smart agriculture monitoring system on a breadboard.

Figure 7 shows the assembled prototype on a breadboard. The ESP32 is connected to the DHT11, soil moisture probe, and MQ7 gas module. Jumper wires and shared grounding support a temporary development setup.

- The DHT11 sensor is connected to GPIO 4 for temperature and humidity measurement.
- The soil moisture sensor is connected to GPIO 34 for analog input.
- The MQ7 gas sensor is connected to GPIO 35 for gas level detection.
- The system is powered through a USB connection.

The breadboard setup is suitable for demonstrations and should be protected from moisture and electrical disturbances before field deployment.

3.1.2 Close-up view of ESP32 interfacing with DHT11, soil moisture, and MQ7 gas sensors.

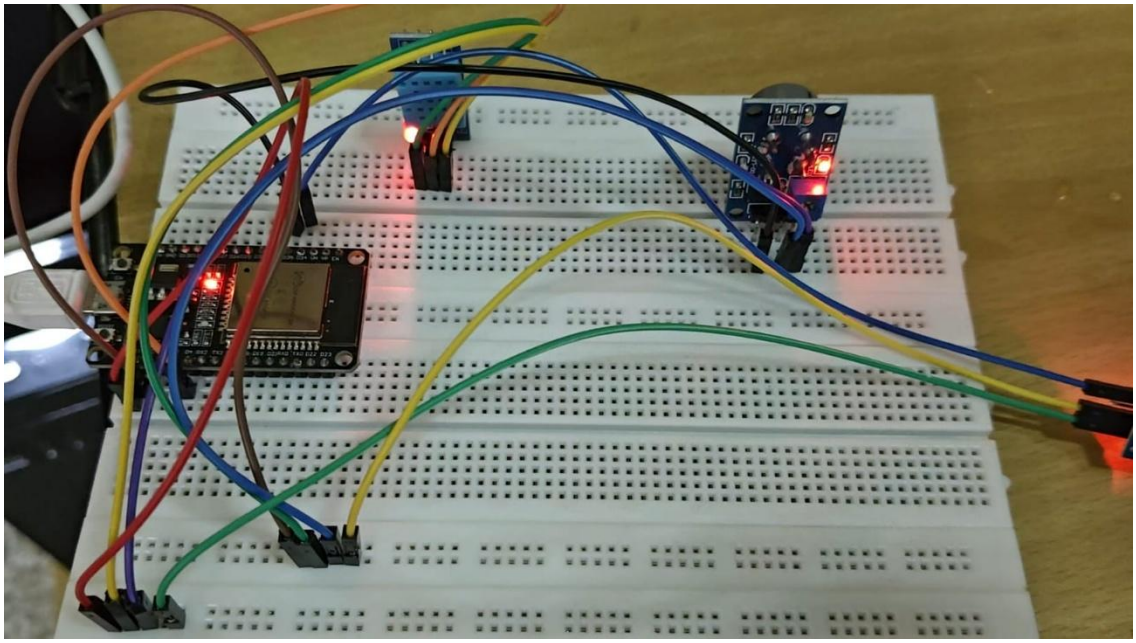


Fig. 8. Close-up view of ESP32 interfacing with DHT11, soil moisture, and MQ7 gas sensors.

Figure 8 shows the interface connections between the ESP32 and the three environmental sensors.

- Analog sensors are connected to input-only ADC pins (GPIO 34 and GPIO 35).
- The DHT11 sensor communicates via a digital signal line.
- All components share a common ground for stable operation.

These connections remain applicable to the new web-dashboard version. The MQ7 output voltage should be checked before connecting it to the ESP32 analog input.

3.1.3 Custom web dashboard for real-time sensor monitoring.

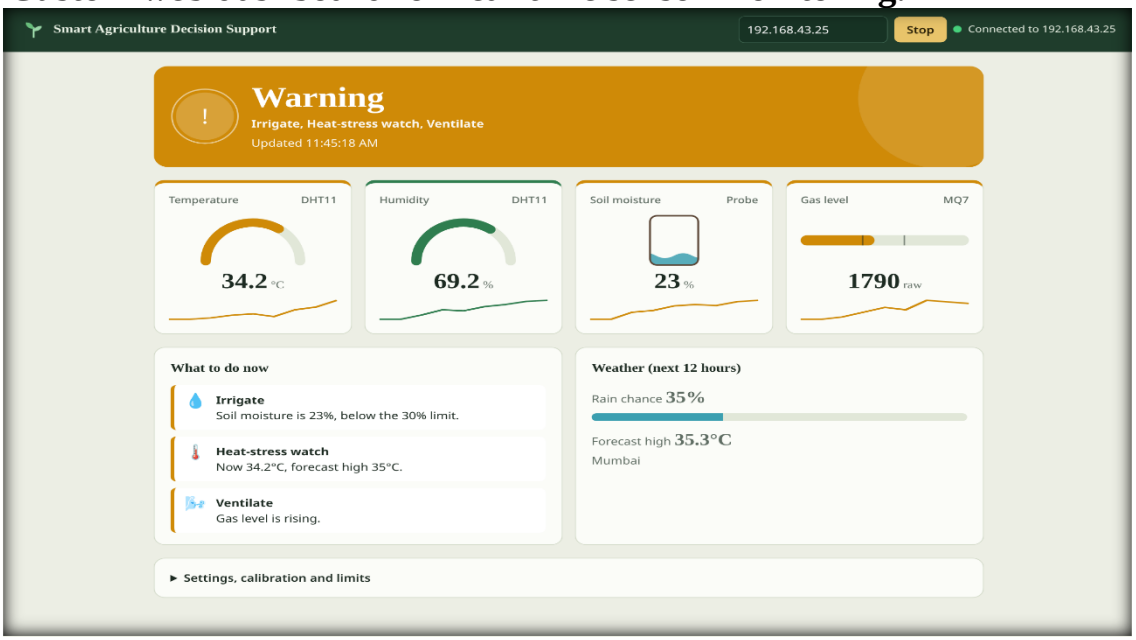


Fig. 9. Custom web dashboard interface (initial connection view)

Figure 9 shows the responsive Smart Agriculture Decision Support web page that replaces the previous Blynk application. The screenshot illustrates the page before a live ESP32 connection has been established.
The web dashboard includes:

- A field for the ESP32 IP address with Connect and Stop controls
- Cards for temperature, humidity, estimated soil moisture, and raw MQ7 readings
- A status banner and a section for threshold-based recommendations
- A weather panel and settings for calibration and warning limits

The browser defaults to polling once every three seconds after connection. Actual field measurements are not represented by the initial-state screenshot.

3.1.4 Dashboard settings for calibration, weather, and decision limits.

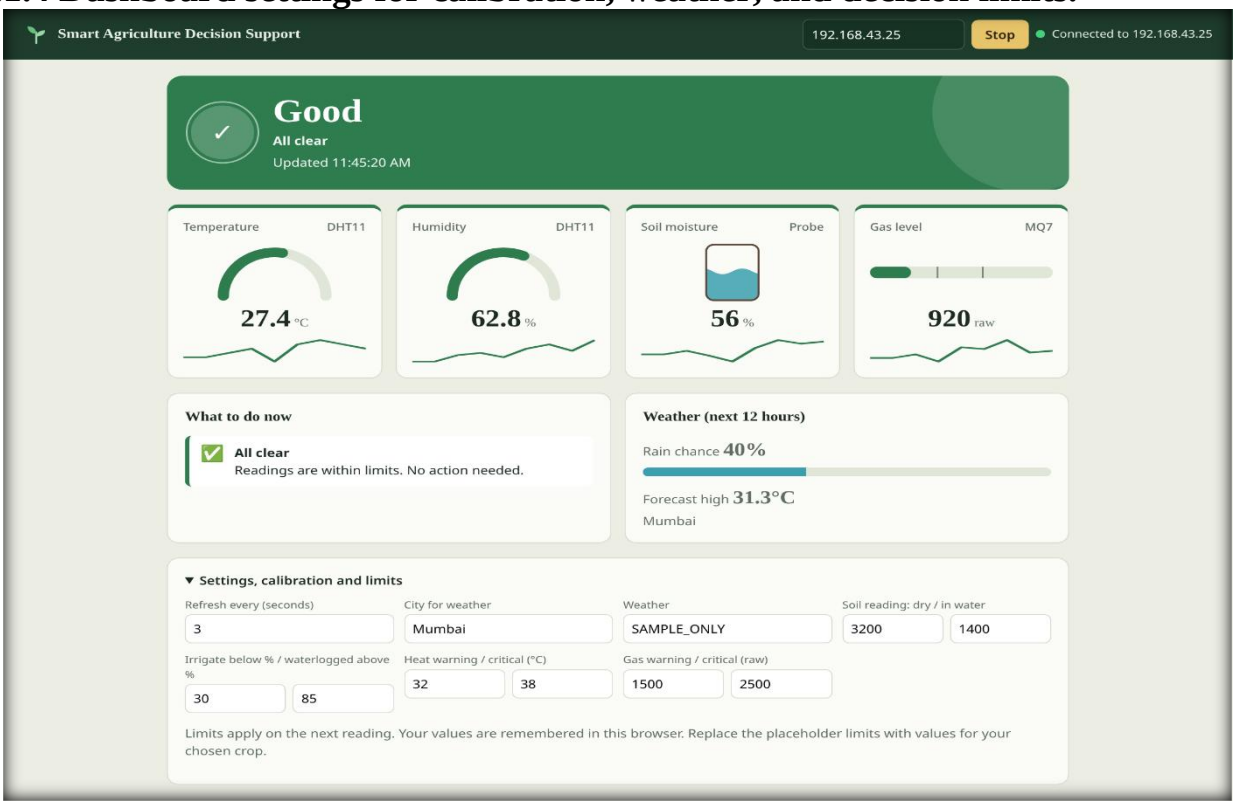


Fig. 10. Dashboard settings and weather configuration

Figure 10 shows the configuration area of the web dashboard rather than the earlier Blynk soil graph.

- Soil percentage uses adjustable calibration readings for dry and wet conditions.
- The dashboard provides irrigation, waterlogging, heat, and raw gas warning thresholds.
- A city and OpenWeatherMap API key may be entered for forecast-based guidance. Settings are saved in the browser; sensor history in the current page is limited to the last 30 readings for mini-trend visualization.

3.1.5 Implementation Summary

The implemented prototype links an ESP32 web server, three sensors, a responsive browser dashboard, and an optional weather forecast. Source review confirms the data format, polling logic, and rule-based recommendations. Field accuracy, yield improvement, and actuation performance have not been independently verified in the supplied materials.

3.2 Code Implementation

The ESP32 firmware is written in Arduino C/C++ and uses the WiFi, WebServer, and DHT libraries. It connects to Wi-Fi, reads the sensors, and returns JSON through the /data endpoint. The HTML/CSS/JavaScript dashboard is maintained as a separate file and fetches this data in the browser.

The sensor refresh interval is approximately two seconds, while the web dashboard requests data every three seconds by default. Sensitive Wi-Fi details are replaced with placeholders in the reproduced code; the weather API is configured and requested through the browser, not the ESP32.

The ESP32 firmware used for this implementation is reproduced below (credentials redacted):

```
#include <WiFi.h>
#include <WebServer.h>
#include "DHT.h"

// ---------- CONFIGURE WIFI ----------
const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";
// ----------------------------------
#define DHTPIN   4
#define DHTTYPE  DHT11
#define SOIL_PIN 34   // ADC1, input only
#define MQ7_PIN  35   // ADC1, input only (use a voltage divider if the module runs at 5 V)
DHT dht(DHTPIN, DHTTYPE);
WebServer server(80);
float temperature = 0, humidity = 0;
int soilRaw = 0, gasRaw = 0;
bool dhtOk = false;
unsigned long lastRead = 0;

int readAnalogAvg(int pin) {
  long sum = 0;
  for (int i = 0; i < 10; i++) { sum += analogRead(pin); delay(5); }
  return sum / 10;
}
```

```
void readSensors() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  dhtOk = !(isnan(h) || isnan(t));
  if (dhtOk) { humidity = h; temperature = t; }
  else Serial.println("DHT11 read failed - check wiring");
  soilRaw = readAnalogAvg(SOIL_PIN);   // higher = drier
  gasRaw  = readAnalogAvg(MQ7_PIN);    // higher = more gas
}

void sendCORS() {
  server.sendHeader("Access-Control-Allow-Origin", "*");   // lets the dashboard page
read this data
  server.sendHeader("Cache-Control", "no-store");
}

void handleData() {
  sendCORS();
  char json[160];
  snprintf(json, sizeof(json),
"{\"temperature\":%.1f,\"humidity\":%.1f,\"soil\":%d,\"gas\":%d,\"dhtOk\":%s}",
          temperature, humidity, soilRaw, gasRaw, dhtOk ? "true" : "false");
  server.send(200, "application/json", json);
}

void handleRoot() {
  sendCORS();
  server.send(200, "text/plain", "ESP32 smart agriculture node is running. Data at
/data");
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(SOIL_PIN, INPUT);
  pinMode(MQ7_PIN, INPUT);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println();
  Serial.print("Connected. Open this in the dashboard: ");
  Serial.println(WiFi.localIP());
  server.on("/", handleRoot);
  server.on("/data", handleData);
```

```
  server.begin();
  readSensors();
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {      // simple auto-reconnect
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    delay(3000);
  }

  server.handleClient();
  if (millis() - lastRead >= 2000) {        // DHT11 needs at least 1 s between reads
    lastRead = millis();
    readSensors();
    Serial.printf("T=%.1f C  H=%.1f %%  Soil=%d  Gas=%d\n", temperature,
humidity, soilRaw, gasRaw);
  }
}
```

Explanation of Code

- The DHT11 is connected to GPIO4 and provides ambient temperature and humidity values. A failed read sets dhtOk to false so the browser can show an appropriate warning.
- The soil moisture probe is sampled on GPIO34; the browser converts its raw value into estimated moisture percentage using the user-adjustable dry and wet readings.
- The MQ7 module is sampled on GPIO35, with ten ADC samples averaged for each reading. The displayed value is raw sensor output and is not a calibrated gas concentration.
- The WebServer handler returns JSON fields temperature, humidity, soil, gas, and dhtOk in response to an HTTP GET request at /data.
- The ESP32 refreshes sensor values approximately every two seconds; the browser periodically fetches the latest cached values at a configurable interval.
- Serial output at 115200 baud displays readings for development checks; the browser separately obtains weather data and evaluates its recommendation rules.

3.3 Future Direction

The current version supports live monitoring and rule-based recommendations, while deliberately keeping the irrigation and ventilation decisions under user control. Several extensions could improve reliability and practical agricultural usefulness.

A future version could use a suitably rated relay, protected power circuit, and water pump for automated irrigation. Any actuation should use verified sensor calibration, manual override, and safe default behavior when Wi-Fi or sensor communication fails.

Measurement quality could be improved with a more accurate temperature-humidity sensor, a corrosion-resistant capacitive moisture probe, documented soil calibration, and proper MQ7 heater cycling and calibration. The ESP32 analog input should be protected from incompatible sensor voltages.

For wider use, the system could add a secured backend or cloud service with authenticated remote access, database storage, device identification, and encrypted transport. MQTT or another IoT messaging protocol could be considered for multiple sensor nodes.

Advanced analytics could make use of retained measurements, crop-specific limits, and verified weather data to support forecasts or improved scheduling. Current trend charts are temporary browser-session views, not a persistent historical database.

Operational improvements could include offline status logs, better network reconnection handling, Over-the-Air firmware updates, environmental protection for the hardware, and safer storage of weather API credentials.

These additions would build on the existing low-cost, browser-based IOE prototype while retaining the separation between measurements, external information services, and user decisions.

4. CONCLUSION

The IoT Based Smart Agriculture Monitoring System demonstrates a functional design for collecting environmental data using an ESP32 connected to DHT11, soil moisture, and MQ7 modules. The principal change from the previous semester is the replacement of the Blynk mobile application with an independent browser dashboard.

The ESP32 firmware hosts a local HTTP /data endpoint that returns JSON measurements. A responsive HTML/CSS/JavaScript dashboard displays current readings, estimated soil moisture, raw gas levels, short trend histories, and a connection indicator. It also provides editable limits for decision support.

The dashboard can integrate a near-term OpenWeatherMap forecast with sensor readings to recommend irrigation, postponement when rain is likely, heat-related caution, or attention to elevated gas readings. These recommendations are rules implemented in JavaScript, not machine-learning predictions or automatic control actions.

The project shows how the IOE concept connects things, data, processes, and people through local sensing, network communication, an external service, and an accessible interface. The scope of verified evidence is limited to the provided code and project materials; quantitative crop improvements and calibrated gas safety performance are not claimed.

Future work can focus on secure remote deployment, reliable data logging, sensor calibration, and carefully designed irrigation automation. Within its current scope, the system provides a clear academic demonstration of web-based smart agriculture monitoring and decision support.

5. REFERENCES

[1] Espressif Systems, “ESP32 Technical Reference Manual.” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] DHT11 Temperature and Humidity Sensor, “Technical Data Sheet.” [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[3] Winsen Electronics, “MQ-7 Carbon Monoxide Sensor Datasheet.” [Online]. Available: https://www.winsen-sensor.com/

[4] Adafruit Industries, “DHT Sensor Library for Arduino.” [Online]. Available: https://github.com/adafruit/DHT-sensor-library

[5] OpenWeather, “5 Day / 3 Hour Forecast API.” [Online]. Available: https://openweathermap.org/forecast5

[6] Espressif Systems, “Arduino ESP32 WiFi Library Documentation.” [Online]. Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

[7] Espressif Systems, “Arduino ESP32 WebServer Library.” [Online]. Available: https://github.com/espressif/arduino-esp32/tree/master/libraries/WebServer

[8] A. Tzounis et al., “Internet of Things in Agriculture: Recent Advances and Future Challenges,” Biosystems Engineering, vol. 164, pp. 31–48, 2017.

[9] Arduino, “Arduino Documentation.” [Online]. Available: https://docs.arduino.cc/

[10] Espressif Systems, “Arduino Core for the ESP32.” [Online]. Available: https://github.com/espressif/arduino-esp32